北京理工大學

专业学位研究生
中 期 检 查 报 告

选题名称 面向多智能体系统的高效通信技术研究

学 号 3220241499

姓 名 柯逸宸

导 师 唐博

类 别 电子信息

领 域 计算机技术

培养单位 珠海校区

年 月 日

目录

1 引言 **3**

1.1 研究背景和意义 3

1.2 研究基础与目标 5

1.3 国内外研究现状及分析 7

2 研究内容 **14**

2.1 研究方案概述 14

2.2 功能模块设计 15

3 实验与分析 **18**

3.1 实验设置 18

3.2 实验结果和分析 20

4 结论 **27**

4.1 研究进度与规划 27

4.2 研究过程中遇到的问题 28

参考文献 **30**

第一章　引言

1.1　研究背景和意义

1.1.1　研究背景

近年来，以 Transformer 架构为基础的大语言模型（Large Language Models，LLMs）[1]，如 OpenAI 的 GPT-4 系列 [2]，以及开源生态中的 LLaMA 系列 [3–5]、Qwen 系列 [6–8] 等，在自然语言理解、逻辑推理与文本生成任务中展现出强大的性能。然而，随着应用场景向高度复杂的现实任务（如软件工程 [9]、科学计算 [10]、具身智能探索等 [11]）拓展，大语言模型的局限性日益凸显 [12]。

为了突破这一局限，学术界提出了赋予大语言模型自主性的智能体（Agent）概念 [12]。智能体以大语言模型强大的逻辑推理能力为基础，包括早期的思维链（Chain of Thought，CoT）[13]、随后提出的思维一致性（Self-Consistency）[14]，以及更复杂的思维树（Tree of Thoughts，ToT）[15] 和思维图（Graph of Thoughts，GoT）[16]。在此基础上，智能体进一步融合了工具调用（Tool Calling/API Use）[17,18]、环境感知与动态反馈机制，以及长周期复杂任务的拆解与规划能力 [19]，现已成为当前人工智能领域的核心发展范式 [20–22]。以 ReAct（Reasoning and Acting）[23] 为代表的方法，将模型的逻辑推理与外部环境执行协同，构建了“思考—行动—观察”的动态闭环，从而显著提升了模型在复杂交互环境中的执行能力 [24]；同时，Reflexion 等基于自然语言反馈的自我修正机制 [25]，以及 Generative Agents、Voyager 等 [26,27] 引入长时记忆架构，有效克服了大语言模型无状态交互的局限，提升了其可靠性与跨轮次持续学习能力。尽管智能体技术发展迅猛，其能力上限仍受制于三个瓶颈：

1. 受限于模型训练数据的日期，其内部权重难以包含最新的知识 [28]。

2. 在上下文窗口限制下，超长输入不仅会带来高昂的计算开销与推理延迟，更易导致关键信息被稀释，引发“迷失在中间（Lost in the Middle）”现象，致使模型丧失对长文本中核心细节的捕捉能力 [29,30]。

3. 单智能体受限于单一的推理视角与预设的角色提示词（Prompt），在应对高度复杂、需多学科交叉分析的任务时，极易陷入局部最优的认知局限 [31]。

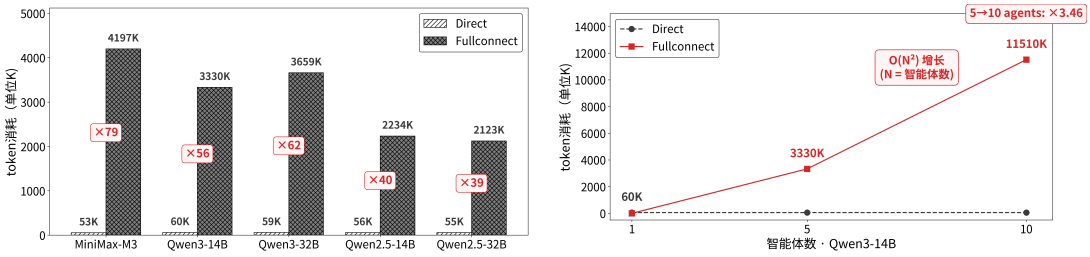
为此，学术界与工业界的研究范式正从单体 Agent 向多智能体系统（Multi-Agent Systems，MAS）演进 [31,32]。MAS 通过模拟人类社会的组织与协作机制，引入了明

确的角色分工（如程序员、测试员、产品经理的职能划分）与交叉验证辩论体系[33,34]。大量前沿研究已经初步论证了 MAS 在复杂场景下的优越性：例如，清华大学提出的 ChatDev 将软件开发过程建模为瀑布流式的多智能体协作网络，大幅降低了代码漏洞率[9]；MetaGPT 通过引入标准化操作程序（SOPs），将复杂任务严密拆解为多角色的流水线作业，有效缓解幻觉[35]；微软的 AutoGen 则提供了一种面向对话编程的可定制化多智能体交互模式，支持多个智能体间的灵活组网[36]。这些研究充分表明，MAS 能够有效突破单智能体的能力天花板，在复杂协同任务中的综合任务成功率、代码通过率与系统鲁棒性均实现了提升。

然而，随着 MAS 能力的提升，其底层通信机制的瓶颈也暴露出来，成为核心障碍：当前主流的多智能体框架[35,36] 在拓扑维度上多依赖静态的拓扑结构，在语义维度上则普遍采用完全传递的策略。这意味着，每当一个智能体生成内容，其原始输出会被全量且未经筛选地复制给所有相关的下游接收节点。这种密集的交互模式引发了两个问题：

1. 通信成本增加：目前模型大多按词元计费。如 1-1所示，一旦智能体数量增多，系统内的消息量就会呈指数级或平方级暴涨，带来巨大的成本压力。

2. 关键信息淹没：冗余上下文会导致 LLM 出现“长上下文注意力衰减”。接收方智能体在处理大量重复和无关的消息时，极易漏掉关键信息，反而拉低了任务的成功率。



(a) 多智能体系统的 token 消耗　　(b) 不同智能体数量下的输入 token

图 1-1 多智能体系统的 token 消耗对比和不同智能体数量下的输入 token 变化

因此，如何减少 MAS 中高昂通信成本，已成为当前大模型多智能体领域需要解决的问题。针对这一问题，本文提出了一种多智能体系统消息压缩框架，旨在通过对冗余语义信息的过滤与压缩，进而降低多智能体协作过程中的 Token 消耗，从而为构建低成本的多智能体通信架构提供可靠的解决方案。

1.1.2 研究意义

针对当前多智能体系统在通信成本与上下文处理上面临的严峻挑战，本文分析了多智能体系统消息压缩框架具有重要的理论价值与实际意义，体现在以下两个层面：

1. 理论意义：本文从语义信息与多智能体通信机制的交叉视角出发，打破了多智能体框架中全量广播的交互范式。通过探索并建立动态、自适应的冗余信息过滤与核心语义保留机制，为解决大语言模型在多轮复杂交互中的“长上下文注意力衰减（Lost in the Middle）”问题提供了新的理论视角。

2. 实际意义：

（1）降低多智能体系统部署成本：通过对传输消息的压缩，大幅减少了模型推理时的冗余 Token 消耗。这不仅直接降低了 API 调用开销，使多智能体系统在工业级、企业级场景中的大规模、低成本部署成为现实。

（2）提升多智能体系统的任务成功率：对消息进行压缩为下游智能体提供了关键信息，有效避免了有效信息被海量历史对话淹没。

1.2 研究基础与目标

1.2.1 研究基础

基于上述背景，在这一部分中，我们将给出大语言模型多智能体系统通信机制中的基础定义，并明确本课题的研究问题。

定义 1 拓扑结构：表示多智能体系统中各智能体（Agent）节点之间信息交互路径的连接关系。

定义 2 常见通信拓扑类型：在多智能体协同系统中，根据信息交互路径的组织方式，拓扑结构主要可以划分为以下四种典型范式：

- **完全拓扑（Full Topology）：**指多智能体系统中任意两个智能体节点间均存在双向的信息传输通路。在该拓扑范式下，每个智能体均接收所有其他智能体的输出内容，并将其全量作为历史上下文。
- **层状拓扑（Layered Topology）：**指智能体按层级结构进行划分，当前层的智能体仅接收并处理上一层智能体的输出信息，适用于具有明显阶段划分的复杂任务。
- **链式拓扑（Chain Topology）：**指信息以串行方式依次传递，从初始输入开始，

每个智能体顺次处理并将结果单向传递给下一个智能体。

· **星状拓扑（Star Topology）：**指引入中心管理智能体，由其统一负责全局信息的收集、分发与汇总，其余外围智能体与中心节点进行交互。

定义 3 语义内容：指在既定的通信拓扑路径上，智能体之间实际传递的信息内容。在大语言模型多智能体系统中，语义内容通常表现为自然语言文本、结构化提示或中间推理状态。

定义 4.1 单轮内对话：指多智能体在单个任务对话周期内，通过协同交互共同完成结果输出的交互架构。本研究所探讨的单轮内对话拓扑结构特指完全拓扑，具体如1-2所示。

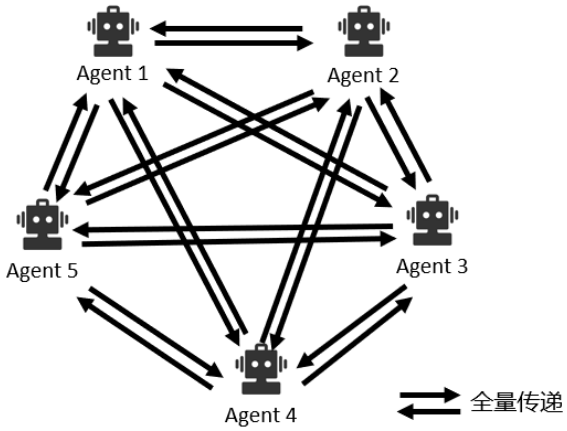


图 1-2 完全拓扑 (Full Topology)

定义 4.2 多轮间通信：指多智能体跨对话轮次进行信息传递与历史信息的协作架构。具体表现为在每一轮交互中，每个智能体均接收并利用上一轮所有智能体的输出信息。

问题 1.1 多智能体通信冗余优化：在多智能体持续协作过程中，随着交互轮次的增加，系统内传递的自然语言上下文会产生严重的语义冗余。如何在约束条件下，寻找最优的过滤机制，从而减少多智能体系统的成本?

围绕上述研究问题，本课题目前已具备以下研究基础:

1. 理论与文献基础：已对 MetaGPT、AutoGen 等主流多智能体协作框架进行了系统性的调研与解构。重点梳理了其底层的交互逻辑、角色分工范式、协同机制以及现有的通信模式。这为本课题重新审视多智能体间的通信瓶颈、进行精确的数学与逻辑建模提供了坚实的理论支撑。

2. 技术储备：在通信网络优化的两个核心维度已掌握关键技术，为算法设计提供

了直接的方法论支撑。在拓扑结构层面，已掌握图生成、图剪枝及网络拓扑搜索等图论与图学习前沿技术。在语义处理层面，已熟练掌握文本信息压缩相关技术，并深入研究了基于困惑度及轻量化模型辅助的提示词过滤与精简策略。

3. 实验环境与资源支持：已具备开展大规模多智能体实验的软硬件条件。目前已获取 Qwen、MiniMax [60] 等主流大型语言模型的 API 权限及计算资源。

1.2.2 研究目标

针对当前多智能体系统（MAS）中由于全量传递所引发的问题，本研究的核心目标是：设计并构建语义驱动的高效多智能体通信架构，在保障或提升任务准确率的前提下，尽可能降低系统的成本。

1.3 国内外研究现状及分析

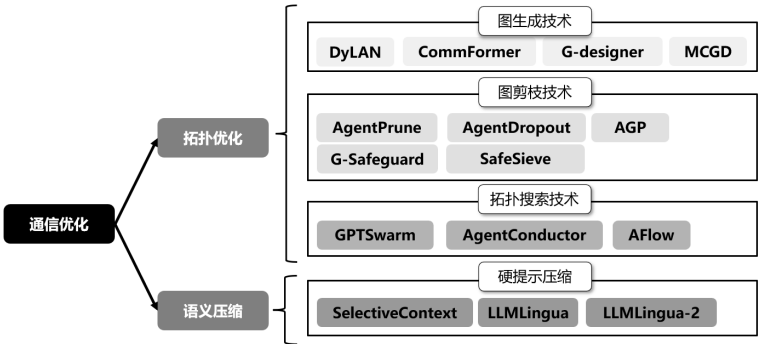


图 1-3 多智能体的两条研究路线

为了应对多智能体系统中日益凸显的通信瓶颈，近年来学术界与工业界在通信机制优化方面进行了大量的探索。依据优化策略切入点的不同，现有研究呈现出两条路线：通信拓扑维度的优化（决定信息传播路径，核心目标是减少不必要的通信连接）与语义维度的消息压缩（决定沿路径传递的信息内容，核心目标是减少单条消息内部的冗余）。本章节将分别对两条路线进行系统梳理。

1.3.1 拓扑维度的优化

在 MAS 中，通信拓扑决定了信息的传播广度。其核心优化目标是通过合理规划信息流转的路径，减少不必要的通信交互，从而在拓扑层面降低 MAS 的词元开销。早期的 MAS 框架主要依赖人工预设的静态拓扑（如完全连接、链式/层状、星状或二分图拓扑等 [9,35–37]）。尽管它们在某些结构化任务中提升了效率，但静态设计缺乏对复杂任务的自适应性，且极易导致通信边数随节点数量呈平方级增长，引发词元消耗剧增。为了克服这一缺陷，当前主流研究方向之一是利用基于图的方法来探索这一问题，

即将智能体视为节点、通信链路视为边。近期的相关研究主要可归纳为三大类：图生成、图剪枝与拓扑搜索[38]。

1. 图生成技术：此类方法旨在通过自适应地生成与任务条件相匹配的图，从头开始构建通信拓扑，主要分为以下两种:

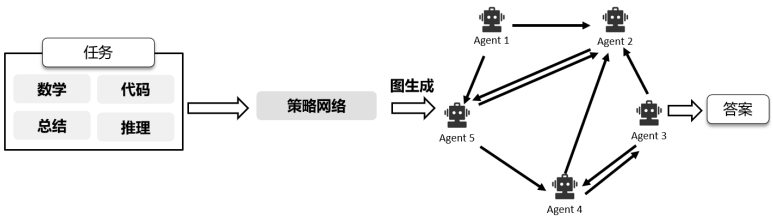


图 1-4 图生成技术示意图

查询驱动的动态生成：DyLAN（Dynamic LLM-Agent Network）[39] 并不预先固定所有智能体之间的通信关系，而是将多智能体协作过程建模为一个随任务推进而变化的前馈式动态网络。其核心机制包括两个阶段：首先在候选智能体集合中进行团队优化，利用 Agent Importance Score 评估不同智能体在当前任务上的边际贡献；随后在任务求解阶段，根据每一轮中智能体输出的有效性动态选择后续参与通信的节点。DyLAN 还引入早停机制，当系统已经形成足够稳定的答案或共识时提前终止多轮交互。因此，它降低成本的方式不是压缩单条消息，而是减少低贡献智能体的参与次数、削减无效讨论轮次，并动态生成更贴合当前查询的通信图。

端到端的网络解码：CommFormer[40] 将多智能体强化学习中的智能体表示为图节点，将通信信道表示为有向边，并通过可微的图结构学习机制直接优化通信拓扑。其关键做法是对离散边选择进行连续松弛，使邻接矩阵能够参与梯度更新，同时在 Transformer 结构中引入关系增强注意力，以建模智能体间的依赖关系。训练目标是在合作任务中最大化团队累计回报，同时满足通信稀疏率或通信预算约束。与全连接广播相比，CommFormer 只保留对策略协同有显著贡献的通信边，从而在保证任务性能的前提下降低通信开销。

G-Designer[41] 面向 LLM 多智能体系统，将每个智能体的基础模型、角色、状态、工具插件等编码为节点特征，并额外引入任务虚拟节点来表达当前查询需求。随后，它使用变分图自编码器生成面向特定任务的通信图。该方法强调性能、复杂度自适应和鲁棒性三方面目标：简单任务生成较稀疏的拓扑，复杂任务则保留更多协作边。其

本质是从任务条件出发解码通信结构，使系统不再依赖人工预设的固定拓扑。

MCGD [42] 则更偏向多智能体强化学习中的协调结构建模。它构建包含连续节点属性与离散边属性的稀疏协调图，并利用图扩散模型学习多智能体间的交互模式和动作生成过程。其优化目标是通过扩散式去噪学习恢复有效的协同行为。虽然它不是典型的 LLM token 通信压缩方法，但其思想同样体现为：只显式建模关键邻居关系，避免全体智能体两两交互带来的结构冗余。

2. 图剪枝技术：此类工作通常从密集的拓扑结构（如全连接）开始，旨在将其修剪为稀疏的、适合任务的流水线，同时降低计算成本。

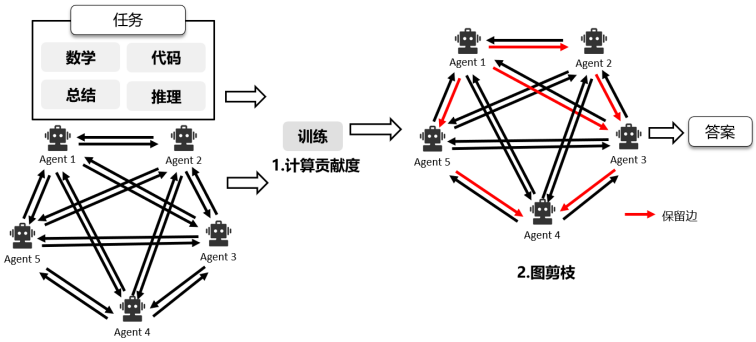


图 1-5 图剪枝技术示意图

硬剔除与软剪枝：AgentPrune [43] 将 LLM 多智能体系统表示为空间-时间通信图：同一轮智能体之间的信息交换为“空间边”，跨轮历史传递为“时间边”。它为这些边学习可剪枝的图掩码，并通过系统效用评估器判断不同边对任务结果的贡献。由于外部 LLM 调用和任务评估通常不可微，AgentPrune 使用策略梯度近似优化掩码参数。最终，它执行一次性硬剪枝，移除低贡献边，从而同时减少同轮通信和跨轮历史上下文传递。AgentDropout [44] 在边剪枝基础上进一步引入节点丢弃。它首先根据智能体贡献度或加权度识别低价值节点，执行 Node Dropout，直接移除部分智能体；随后再通过 Edge Dropout 剪除剩余智能体之间的低贡献通信边。由于删除一个智能体会同时减少该智能体的输入 prompt、输出 completion 以及后续通信边，因此 AgentDropout 的成本收益通常比单纯剪边更直接。SafeSieve [45] 采用渐进式软剪枝策略。它首先利用 LLM 语义评估和角色互补性为通信边初始化重要性分数，再结合历史任务表现不断修正边权。与简单 top-k 删除不同，SafeSieve 使用聚类机制保持核心功能簇的连通性，避免剪枝破坏系统协作结构。因此，它更强调“逐步削弱和删除冗余边”，在保持任务性能和鲁棒性的同时降低 token 与部署成本。AGP

（Adaptive Graph Pruning）[46] 同时进行节点级硬剪枝和边级软剪枝。其训练过程通常分为两步：先采样不同规模的子图，收集高性能通信结构作为监督信号；再训练 GNN 根据任务表示和智能体嵌入输出节点 mask 与边权。硬剪枝决定哪些智能体被启用，软剪枝决定启用智能体之间的信息流强度。这样，AGP 能针对不同查询生成不同规模的稀疏拓扑，而不是对所有任务使用同一套固定通信结构。

基于安全机制的剪枝：G-Safeguard [47] 的主要目标是提升 LLM 多智能体系统的安全性。它构建 multi-agent utterance graph，将智能体发言、交互关系和传播路径纳入图神经网络检测框架，用于识别受到 prompt injection、memory attack 或 tool attack 影响的异常节点与异常话语。随后系统可以隔离危险智能体或剪断其高风险外发边，阻止恶意信息继续扩散。它的直接收益是安全鲁棒性，通信成本降低更多是拓扑干预带来的副效应。

3. 拓扑搜索技术：拓扑搜索通过在智能体算子和通信边构成的庞大图空间中进行探索，以寻优出高效的流水线或通信子图。这类方法通常结合强化学习或网络架构搜索算法。

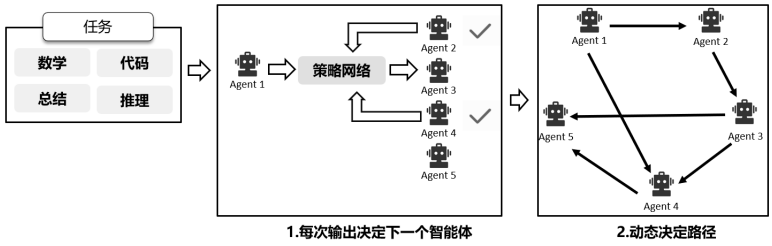


图 1-6 拓扑搜索技术示意图

基于强化学习的连续搜索：GPTSwarm [48] 将语言智能体系统统一建模为可优化图：节点可以是 LLM 调用、工具或数据处理函数，边表示信息流。它为候选边分配可学习的采样概率，在无环图空间中采样候选拓扑，并通过 REINFORCE 等策略梯度方法根据任务效用更新边概率。随着训练推进，低收益边的采样概率趋近于 0，高收益边被保留下来，从而形成任务驱动的稀疏通信图。

AgentConductor [49] 面向竞赛级代码生成任务，训练一个 LLM 为每个问题动态生成 YAML 格式的分层 DAG 拓扑。图中节点是不同角色的代码智能体，边表示跨层依赖关系，同层节点可并行执行。系统会根据编译、运行和测试反馈多轮调整拓扑。其训练包含监督微调和强化学习，奖励信号同时考虑结构合法性、代码执行结果和难

度感知的图密度，因此简单题倾向于生成浅层少边图，复杂题才启用更密集的协作结构。

多样化的搜索策略：AFlow[50] 将 agentic workflow 表示为可执行代码中的算子组合与依赖关系。它预定义 Generate、Review、Revise、Ensemble、Test、Programmer 等工作流算子，并使用蒙特卡洛树搜索在候选工作流空间中探索。每个搜索节点对应一个完整工作流，系统通过验证集执行结果回传分数，逐步扩展更有希望的结构。AFlow 不以显式边稀疏正则为核心，而是通过自动搜索找到更短、更有效的工作流程序，从而减少人工设计和冗余调用。

MASS（Multi-Agent System Search）[51] 认为多智能体系统的性能同时受 prompt 和 topology 影响，因此采用交错优化流程：先优化模块级 prompt，再搜索工作流拓扑，最后进行全局 prompt 优化。其拓扑空间由 Aggregate、Reflect、Debate、Summarize、Tool-use 等方式构成。MASS 会评估不同模块相对于单智能体基线的贡献，对低收益或负收益模块进行影响力剪枝，再在剩余空间中采样和验证候选拓扑。

MaAS（Multi-agent Architecture Search via Agentic Supernet）[52] 提出“智能体超网”思想，将多智能体架构表示为可按查询采样的层级子网。控制器根据 query embedding 和上一层已选算子，逐层选择 Generate、SelfRefine、Programmer、Ensemble 等候选算子。其训练使用策略梯度式目标，效用函数通常同时考虑任务得分和调用成本。推理时，每个查询只激活超网中的少量算子路径，并可结合早停机制降低 token 和调用开销。

DynaSwarm[53] 则在 GPTSwarm 式参数化通信图基础上进一步引入样本级动态选择。它先通过 Actor-Critic 学习多个高性能候选图，再训练 graph selector 根据输入样本选择最合适的拓扑。使得系统不再依赖一个全局静态最优图，而是针对不同任务实例路由到不同稀疏候选结构，在保持较低选择延迟的同时提升拓扑适配性。

1.3.2 语义维度的压缩

除了在通信拓扑层面减少冗余交互外，在语义维度直接对智能体传递的上下文消息进行压缩，是降低大型语言模型（LLMs）推理成本、提升内存利用率的另一项重要手段。与拓扑优化关注“哪些智能体之间需要通信”不同，语义压缩更关注“沿通

信链路传递什么内容”，其核心思想是在尽量保持任务相关语义不丢失的前提下，减少上下文中的冗余词元、重复表述和低价值信息。在实现这一目标的诸多策略中，硬提示词压缩技术（Hard Prompt Methods）作为一种无需修改模型底层架构、能够直接适配黑盒 LLM 接口的实用方案，受到了学术界和工业界的广泛关注。本节将重点探讨此类技术。

硬提示词压缩旨在通过对原始文本进行修改，生成人类可读的、更为精简的自然语言序列。与软提示压缩将上下文映射为不可读的连续向量不同，硬提示词压缩保留了自然语言形式，因此更便于人工检查、跨模型迁移以及在多智能体通信场景中复用。根据压缩机制的不同，主流研究主要可划分为以下两个方向[54]：

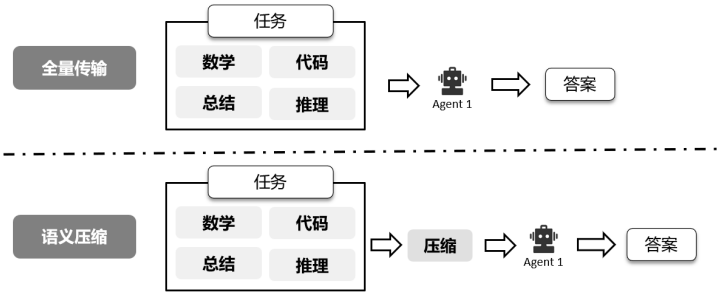


图 1-7 压缩示意图

1. 基于过滤与剪枝 (Filtering)：此类方法通过评估词元、短语或句子的重要性，剔除冗余和低信息量的部分，并在保持原文顺序的基础上生成压缩后的上下文。早期代表性工作 SelectiveContext[55] 利用较小规模的因果语言模型计算词元的自信息或困惑度，将低自信息内容视为更容易由上下文预测、因而冗余度更高的部分，再按照词元、短语或句子粒度进行过滤。该方法无需额外训练，能够直接应用于黑盒 LLM，但由于其重要性评估主要依赖语言统计特征，可能误删对特定任务有用但语言上并不突出的信息。LLMLingua[56] 进一步提出粗到细的压缩框架，首先通过预算控制器在指令、示例和问题等不同提示词组成部分之间分配压缩率，再利用小模型估计词元困惑度或信息熵，逐步保留高信息量词元。相比简单过滤，LLMLingua 考虑了提示词内部不同模块的重要性差异，并通过迭代式压缩缓解局部删除造成的上下文断裂问题，因此能够在较高压缩率下保持较好的下游任务性能。

后续的 LLMLingua-2[57] 则引入了数据蒸馏机制，将压缩任务从基于困惑度的无监督筛选转化为词元级二分类任务。具体而言，该方法先利用强模型生成抽取式压缩结果，再通过自动对齐为原始文本中的词元标注“保留”或“删除”标签，并

训练轻量级双向编码器预测每个词元的重要性。推理时，模型按照保留概率选择满足压缩预算的词元，并保持其原始顺序。由于双向编码器能够同时利用左右上下文，LLMLingua-2 在压缩速度、忠实度和跨任务泛化方面均优于单纯依赖困惑度的方法。进一步地，TACO-RL [58] 将强化学习引入过滤式压缩过程，将每个词元的保留或删除视为动作，并根据下游任务表现给予奖励。例如，在问答任务中可依据答案 F1 值评估压缩效果，在摘要任务中可依据生成文本与参考文本的相似度计算奖励。通过这种方式，TACO-RL 能够根据具体任务目标动态调整剪枝策略，使压缩器更倾向于保留与当前查询或任务评价指标直接相关的信息，从而弥补任务无关过滤方法在语义选择上的不足。

2. 基于释义与重写 (Paraphrasing)：与简单的删除词元不同，此类方法利用语言模型强大的生成与概括能力，主动将冗长的上下文重写为更紧凑的自然语言表达。其核心优势在于能够跨越句子边界进行信息重组，将分散在原文不同位置的关键信息合并为更短的表述，从而避免纯过滤方法可能造成的语义割裂或可读性下降。例如 Nano-Capsulator 框架 [59] 不再直接从原始提示词中选择词元，而是训练一个生成式压缩模型，将长提示词封装为更短的 Capsule Prompt。该模型在训练过程中同时考虑语义保持和下游任务效用：一方面要求压缩后的自然语言表示尽可能保留原始提示词的核心含义，另一方面又通过目标 LLM 在相关任务上的输出表现约束压缩结果，使其不仅短，而且仍然能够有效引导模型完成任务。

与过滤式方法相比，基于释义与重写的压缩能够生成更加流畅、连贯且人类可读的压缩文本，尤其适合处理包含重复解释、长链式推理示例或多轮对话历史的上下文。在多智能体系统中，这类方法可用于将某一智能体的长篇推理过程、观察记录或工具调用结果重写为面向后续智能体的短摘要，从而减少跨智能体传递的词元数量。然而，这类方法也存在一定风险：由于压缩过程依赖生成模型，可能出现关键信息遗漏、语义偏移甚至幻觉。因此，在高精度任务中，重写式压缩通常需要结合任务反馈、事实一致性检测或保守的压缩率设置，以在压缩效率与语义可靠性之间取得平衡。

第二章　研究内容

在这一章节中，我们将详细阐述如何将通信优化嵌入多智能体系统任务执行流程，并解析各个功能模块的具体设计。

2.1　研究方案概述

在这一章节中，我们将描述当前多智能体执行的流程，并给出消息压缩框架的初步设计。

2.1.1　多智能体系统任务执行流程

多智能体系统通过角色分工与协作推理来完成目标。如图 2-1所示，其标准任务执行流程主要包含以下四个核心环节：

1. **任务输入：**接收并标准化外部任务，为智能体构建结构化的上下文环境。

2. **协作推理：**依托预设的通信拓扑，各智能体结合自身角色与前驱节点的历史信息进行多轮次推理，并持续传递中间结果。

3. **结果聚合：**采用投票或共识决策机制，对各智能体的候选答案进行汇总，输出最终可靠结果。

4. **性能评测：**系统性评估模型在任务准确率及通信成本（如词元消耗、推理延迟）上的综合表现。

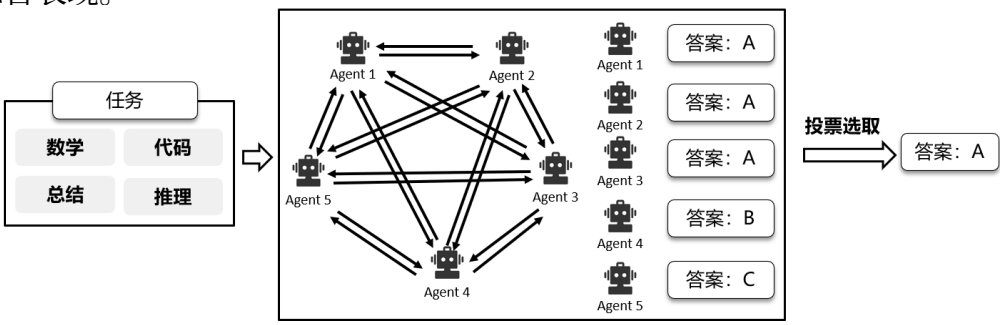


图 2-1 多智能体系统标准任务执行流程示意图

2.1.2　消息压缩框架执行流程

为解决多智能体协作中的信息冗余问题，本课题在智能体与智能体之间的通信中引入了**消息压缩框架**。该框架从角色、节点和时序三个维度对通信内容进行压缩。

具体执行流程包含以下三个核心步骤：

1. **角色感知压缩：**针对接收方信息需求的不对称性，根据下游角色的具体职责，过滤前驱智能体消息中的无关推理内容，从而使下游智能体能够获得与自身任务更相关

的信息。

2. **节点感知聚合：**针对多个前驱节点可能产生相似推理内容的问题，将不同节点传来的相似推理过程进行去重，避免相同观点在下游智能体上下文中重复出现，从而减少冗余消息传递。

3. **分歧感知压缩：**针对多轮对话记录过长的问题，通过提取历史轮次中的共识与分歧内容，压缩上一轮次传递的信息，并保留有效信息供本轮智能体参考。

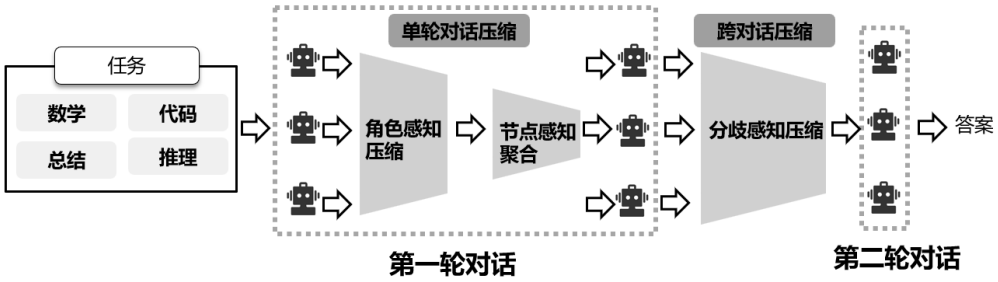


图 2-2 消息压缩框架

2.2 功能模块设计

本节介绍消息压缩框架的三个核心模块：角色感知消息压缩、节点感知聚合和时分歧感知压缩。三者分别处理多智能体通信中的三类冗余：第一类是发送方推理内容与接收方角色无关，第二类是多个前驱智能体给出了相似推理，第三类是跨轮历史持续累积导致上下文过长。为避免符号堆叠，本文仅保留必要定义，并重点说明各模块的输入、处理逻辑和输出结果。

设多智能体系统的通信图为 $G = (V, E)$。在第 t 轮中，发送方智能体 u 传递给接收方智能体 v 的消息表示为

$$m_{u \to v}^{(t)} = (r_u^{(t)}, a_u^{(t)}),$$

其中 $r_u^{(t)}$ 表示推理内容，$a_u^{(t)}$ 表示最终答案。本文的压缩操作主要作用于推理内容 $r_u^{(t)}$，不直接改写答案字段，从而避免压缩过程破坏最终判断。

2.2.1 角色感知压缩模块

角色感知压缩模块解决的是“消息是否适合当前接收方”的问题。在多智能体系统中，不同智能体承担的职责不同。例如，负责规划的智能体更需要任务拆解和约束信息，负责验证的智能体更关注候选答案、错误风险和反例。如果发送方的完整推理被直接转发给所有下游节点，接收方会同时看到大量与自身职责无关的内容，既增加 token 成本，也容易稀释真正有用的信息。

该模块的处理流程分为三步。首先，将发送方的推理文本按句子切分，得到若干候选推理片段。其次，将每个句子与接收方角色描述进行语义匹配，匹配分数由文本向量的余弦相似度给出：

$$\mathrm{sim}(s, d_v) = \cos(\phi(s), \phi(d_v)),$$

其中 s 表示候选句子，d_v 表示接收方角色描述，$\phi(\cdot)$ 表示文本编码函数。最后，系统根据同一条消息内部的相似度分布自动确定保留阈值，只保留与接收方角色更相关的句子；若保留文本仍超过角色预算，则按原始顺序截断到预算范围内。

该模块的输出仍然是一条结构化消息：推理字段被替换为筛选后的短文本，答案字段保持不变。这样做的好处是，接收方能够获得更贴近自身职责的上下文，同时不会因为压缩推理过程而丢失发送方给出的最终答案。

2.2.2 节点感知聚合模块

节点感知聚合模块解决的是“多个前驱消息之间是否重复”的问题。对于同一个接收方节点，多个上游智能体可能从不同角度得出相近结论，也可能在推理过程中反复陈述同一事实。如果将这些内容直接拼接，下游节点会接收到大量重复表达，导致上下文变长，但信息增量有限。

该模块会先收集接收方节点所有前驱智能体的压缩后推理句子，并为每个句子保留来源角色标签。随后，系统按照句子出现顺序逐一检查候选内容：若当前句子与已保留句子的语义相似度高于冗余阈值，则认为它提供的信息已经被覆盖，不再重复加入；若相似度低于阈值，则将其作为新的有效信息保留。本文实现中，冗余判定阈值取 0.70，用于在“去重”和“保留多样观点”之间取得平衡。

聚合后的推理内容由若干带角色标签的句子组成。角色标签的作用是保留信息来源，使下游智能体不仅知道“有哪些观点被保留”，也能知道“这些观点来自哪个角色”。对于多个前驱节点给出的答案，模块采用简单投票生成聚合答案；若出现票数接近或无法明确多数的情况，则保留多个候选答案及其来源，交由后续节点继续判断。

通过该模块，系统不再把多个前驱消息拼接，而是先做跨节点去重和来源标注。这样既能减少重复 token，也能提高下游上下文的信息密度。

2.2.3 分歧感知压缩模块

分歧感知压缩模块解决的是“跨轮历史如何保留”的问题。多智能体协作通常需要多轮执行，如果每一轮都完整保存所有智能体的推理和答案，历史记忆会快速膨胀。过长的历史不仅增加推理成本，还可能使模型在后续轮次中难以抓住当前最关键的共识和分歧。

该模块在每一轮协作结束后运行。系统首先收集本轮所有智能体的答案，并对答案文本进行规则化处理，例如去除多余空格、统一大小写、去掉常见答案前缀和标点。经过规则化后，表达相同含义的答案会被归入同一个答案组。基于答案组数量，系统生成两类摘要。

当所有智能体的答案归入同一组时，说明本轮协作已经形成共识。此时摘要只保留共识答案，以及最能代表该结论的简短理由。代表理由通常来自推理内容较完整的智能体，并只保留核心句子，避免把完整推理链再次写入历史。

当答案被分成多个组时，说明智能体之间仍存在分歧。此时摘要会记录每个候选答案、支持该答案的智能体数量，以及对应的简短理由。这样，下一轮智能体能够直接看到当前争议点，而不需要重新阅读上一轮所有冗长输出。

最终生成的摘要会写入各智能体的历史记忆，作为下一轮推理的压缩上下文。该模块只依赖规则化文本处理、答案分组和简短摘要模板，不额外调用大语言模型，因此能够在控制成本的同时保留跨轮协作中最重要的信息。

第三章　实验与分析

3.1　实验设置

目前，消息压缩框架实现已完成，我们在一些数据集中进行了实验，在本章节中将会展示已经完成的实验结果，并给出对应结果的分析。

3.1.1　数据集

为全面客观地评估模型能力，本文从多智能体系统领域广泛认可的 6 个数据集，涵盖综合知识、数学推理和代码生成三个核心能力维度。具体任务特征与选取依据如表 3-1 所示。

任务类型	数据集名称	描述	样本数量
综合知识	MMLU [61]	涵盖 57 个不同学科的综合性基准	153
数学推理	GSM8K [62]	小学数学应用题数据集	1319
	AQuA [63]	复杂的代数应用题	254
	MultiArith [64]	多步算术运算	600
	SVAMP [65]	语义和扰动生成的数学题	1000
代码生成	HumanEval [66]	生成完整 Python 代码	164

表 3-1 实验评测数据集概览

3.1.2　基座模型

为了保证实验结论的客观性与普适性，本文选取了三个主流大语言模型作为多智能体的基座。这三个模型在 API 调用成本存在显著差异。具体模型计费标准如表 3-2 所示。

模型名称	输入价格（千 tokens）	输出价格（千 tokens）
Qwen2.5-14B-Instruct	¥0.0007	¥0.0007
Qwen3-14B	¥0.0005	¥0.0020
MiniMax-M3	¥0.0021	¥0.0084

表 3-2 API 调用单价对比表

Qwen2.5-14B-Instruct：作为 24 年开源指令模型的代表，其在通用问答、指令遵循与文本生成等任务中表现出较高的稳定性。本研究将其设为基础对照组，旨在与 Qwen3-14B 形成严格的控制变量（参数量级一致）。通过引入该对照组，重点考察在缺乏新一代推理与指令优化机制的前提下，消息压缩策略对任务生成质量、词元（Token）消耗及调用成本的具体影响。

Qwen3-14B：作为中等规模参数开源模型的代表，该模型在推理效率、部署成本与基础性能之间实现了较优的平衡。结合表 3-2 数据可知，其 API 调用成本相对低廉。选取该模型旨在探究消息压缩策略在低成本、常规算力约束下，对实际任务效能及经济成本的优化潜力。

MiniMax-M3：引入该闭源商业模型旨在拓展实验基座的架构多样性，并构建高成本模型对照场景。鉴于其 API 调用费率显著较高（输入与输出单价约为 Qwen3-14B 的 4 倍以上），在处理长文本任务时极易产生高昂的边际成本。选取该模型旨在验证消息压缩策略在面对高资源消耗型商业大模型时，能否有效降低计算冗余并实现显著的成本缩减。

在所有实验中，均采用相同的系统提示词、智能体数量、超参数以及数据集划分，以确保实验结果的可比性。

3.1.3 对比方法

实验设置了四类对比方法。首先是 Direct 方法，即由单个模型直接完成任务，不引入多智能体之间的消息传递，作为任务基础性能与最低通信成本的参照。其次是 Fullconnect 完全连接基线，上游智能体生成的原始输出直接、完整地透传给所有下游智能体，不进行任何截断，从而代表系统通信开销的上限。再次是 AgentPrune 方法基线，作为当前多智能体系统拓扑优化的代表性方法，该基线对智能体间的通信边进行剪枝。最后是本文方法，即引入本文设计的消息压缩模块，在智能体交互前对冗长的推理和弱相关背景进行压缩，仅精准保留关键信息。

3.1.4 评估指标

本文主要从任务完成质量与通信成本两个维度，选取了以下两项核心评价指标：

准确率 (Accuracy)：衡量模型完成任务的有效性。针对 MMLU 及四项数学推理数据集（GSM8K、AQuA、MultiArith、SVAMP），提取模型最终输出的答案与标准

答案进行精确匹配以计算正确率；对于 HumanEval 代码生成数据集，则以测试用例的通过率（Pass@1）作为准确率进行评估。

词元花费 (Token Cost)：衡量多智能体通信过程中的经济开销。具体统计系统在完成单次完整任务时，所消耗 token 的价格。

3.2 实验结果和分析

基于上述实验设置，本文从推理性能和推理成本两个方面对实验结果进行分析。

3.2.1 推理性能分析

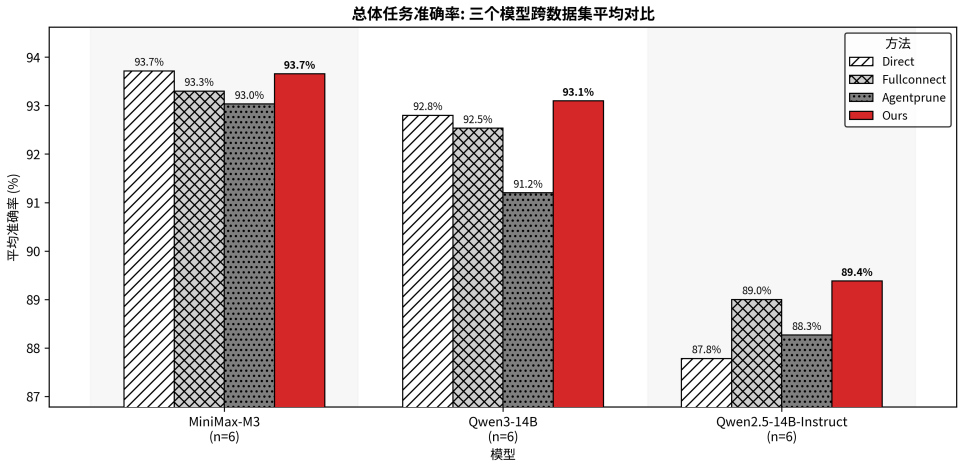


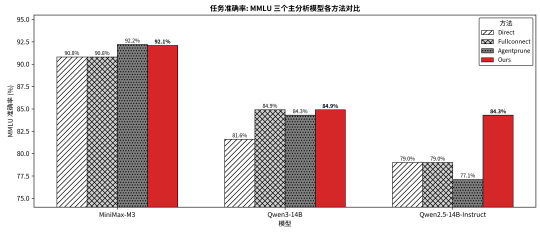
图 3-1 六个数据集在不同模型下的平均准确率对比

图 3-1展示了三个主要模型在不同方法下的跨数据集平均准确率。整体来看，本文方法在 Qwen3-14B 和 Qwen2.5-14B-Instruct 上均取得最高平均准确率，在 MiniMax-M3 上与 Direct 方法基本持平，同时高于 Fullconnect 和 Agentprune。这说明本文方法在不同模型能力水平下均具有较好的稳定性，尤其在中小规模模型上能够带来更明显的性能收益。

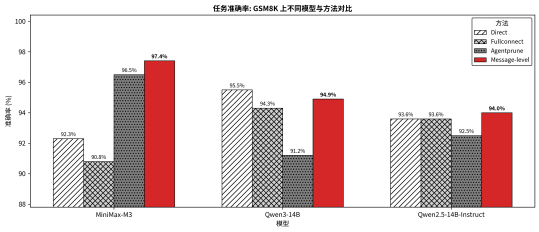
在 Qwen3-14B 上，本文方法的平均准确率达到 93.10%，高于 Direct 的 92.80%、Fullconnect 的 92.53% 和 Agentprune 的 91.20%。在 Qwen2.5-14B-Instruct 上，本文方法的平均准确率为 88.46%，高于 Direct 的 86.62%、Fullconnect 的 88.08% 和 Agentprune 的 87.42%。在 MiniMax-M3 上，Direct 方法的平均准确率为 93.72%，本文方法为 93.66%，二者差距仅为 0.06 个百分点，基本处于同一水平；同时，本文方法仍高于 Fullconnect 和 Agentprune。

为了避免总体平均掩盖不同任务类型的差异，下面进一步按数据集展开分析。总体上，本文方法在 MMLU、GSM8K、MultiArith 和 SVAMP 上表现较强，在 AQuA

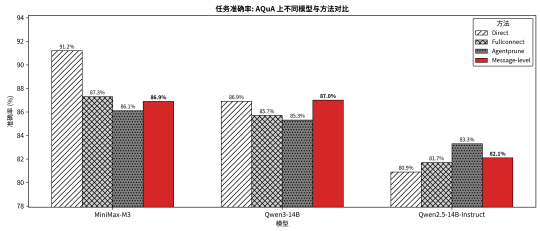
和 HumanEval 上则呈现出更明显的任务依赖性，说明消息级协作更适合知识理解和数学推理类任务，而在多项选择干扰项较强或代码生成需要执行反馈的任务中仍有改进空间。



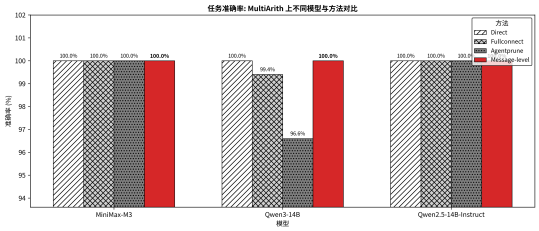
(a) MMLU 准确率



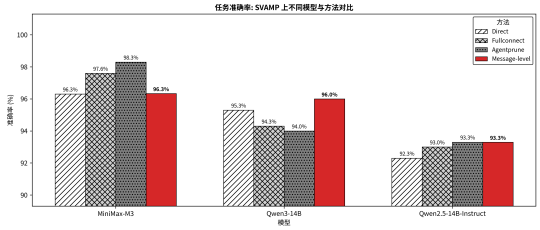
(b) GSM8K 准确率



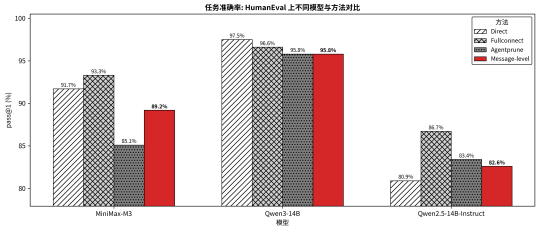
(c) AQuA 准确率



(d) MultiArith 准确率



(e) SVAMP 准确率



(f) HumanEval 准确率

图 3-2 各数据集上不同模型与方法的准确率对比

MMLU 在 MMLU 数据集上，本文方法整体表现较为稳定。在主分析的三个模型中，本文方法的平均准确率为 87.10%，高于 Direct 的 83.80%、Fullconnect 的 84.90% 和 Agentprune 的 84.53%。其中，Qwen2.5-14B-Instruct 的提升最明显，本文方法达到 84.30%，相比 Direct 和 Fullconnect 均提升 5.30 个百分点，相比 Agentprune 提升 7.20 个百分点。Qwen3-14B 上，本文方法达到 84.90%，与 Fullconnect 持平，并高于 Direct 和 Agentprune。MiniMax-M3 上，本文方法为 92.10%，与 Agentprune 的 92.20% 基本持平，并高于 Direct 和 Fullconnect。

GSM8K 在该数据集上，本文方法在两个可用模型上的平均准确率为 96.16%，高于 Direct 的 93.90%、Fullconnect 的 92.55% 和 Agentprune 的 93.85%。其中，MiniMax-M3 使用本文方法达到 97.41%，分别高于 Direct、Fullconnect 和 Agentprune 5.11、6.61 和 0.91 个百分点，说明消息级协作能够增强复杂算术推理中

的答案鲁棒性。Qwen3-14B 上，Direct 方法为 95.50%，略高于本文方法的 94.90%；但本文方法仍高于 Fullconnect 和 Agentprune，说明在该模型上协作信息需要进一步控制以避免轻微干扰。

AQuA 在 AQuA 数据集上，结果更具任务依赖性。三个模型上，本文方法的平均准确率为 85.33%，低于 Direct 的 86.33%，但高于 Fullconnect 和 Agentprune 的 84.90%。分模型看，Qwen3-14B 使用本文方法达到 87.00%，为该模型下最高结果，相比 Direct、Fullconnect 和 Agentprune 分别提升 0.10、1.30 和 1.69 个百分点；Qwen2.5-14B-Instruct 上，本文方法为 82.10%，高于 Direct 和 Fullconnect，但低于 Agentprune；MiniMax-M3 上，Direct 方法达到 91.20%，明显高于其他协作方法。这说明多项选择推理任务中，额外协作信息可能同时带来有用线索和干扰信息，需要更强的任务自适应筛选。

MultiArith 在 MultiArith 数据集上，各方法整体接近饱和。本文方法在三个主分析模型上均达到 100.00%，与 Direct 的平均表现持平，并高于 Fullconnect 的 99.80% 和 Agentprune 的 98.87%。其中，Qwen3-14B 上 Fullconnect 为 99.40%、Agentprune 为 96.60%，而本文方法保持 100.00%。

SVAMP 相比 MultiArith，SVAMP 更强调语义理解和问题变体。三个主分析模型上，本文方法的平均准确率为 95.21%，与 Agentprune 的 95.20% 基本持平，并高于 Direct 的 94.63% 和 Fullconnect 的 94.97%。其中，Qwen3-14B 使用本文方法达到 96.00%，高于 Direct、Fullconnect 和 Agentprune；Qwen2.5-14B-Instruct 上，本文方法达到 93.30%，与 Agentprune 持平；MiniMax-M3 上，Agentprune 达到 98.30%，高于本文方法的 96.33%。

HumanEval 在 HumanEval 数据集上，本文方法并未取得最优平均结果。三个主分析模型中，Fullconnect 的平均准确率为 92.20%，Direct 为 90.03%，本文方法为 89.20%，Agentprune 为 88.10%。分模型看，MiniMax-M3 和 Qwen2.5-14B-Instruct 上 Fullconnect 表现最好，Qwen3-14B 上 Direct 表现最好；本文方法在 MiniMax-M3 上高于 Agentprune，在 Qwen3-14B 上与 Agentprune 持平，在 Qwen2.5-14B-Instruct 上高于 Direct 方法 1.70 个百分点但低于 Fullconnect。该结果说明，对于代码生成任务，仅依赖消息级协作可能不足以替代代码执行、单元测试

或错误反馈机制，后续应将消息筛选与可执行验证结合起来。

综合六个数据集的细分结果可以看出，本文方法的优势更适合概括为在多数任务和模型上提升或保持准确率，而不是在所有单项任务上绝对最优。尤其在 MMLU、GSM8K、MultiArith 和 SVAMP 上，本文方法体现出较好的稳定性；而在 AQuA 和 HumanEval 上，不同任务类型对协作信息的需求差异更明显，说明后续需要根据任务类型、模型能力和推理不确定性动态调整消息筛选强度。

3.2.2 成本分析

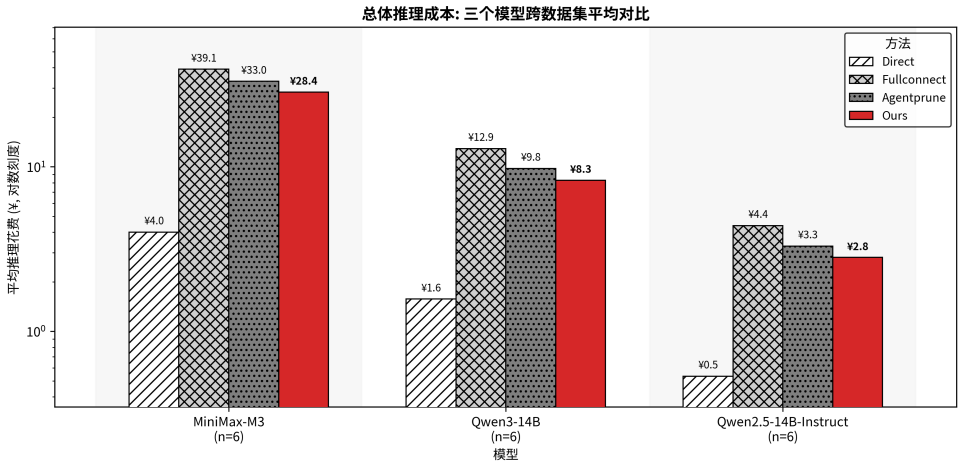
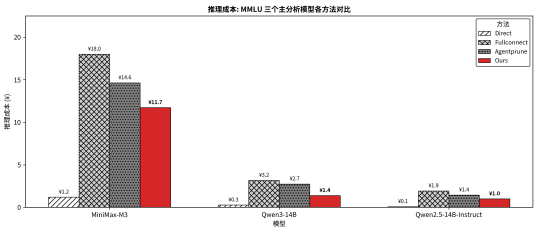


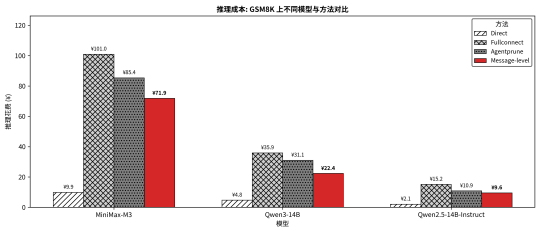
图 3-3 六个数据集在不同模型下的平均推理成本对比

图 3-3展示了三个主要模型在不同方法下的跨数据集平均推理成本。Direct 方法由于不引入智能体间通信，具有最低推理成本；Fullconnect 方法虽然提供了最充分的信息交互，但也带来了最高的通信开销。相比之下，本文方法通过消息粒度的信息筛选减少冗余通信，只保留对最终推理有帮助的关键信息，因此在多数数据集上能够显著降低推理成本。

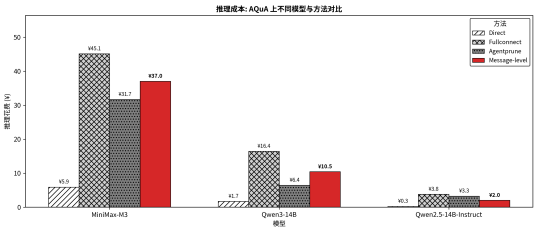
相比 Fullconnect，本文方法在 MMLU 上的平均成本降低约 38.83%，在 GSM8K 上降低约 31.10%，在 SVAMP 上降低约 36.19%，在 MultiArith 上降低约 34.31%，在 HumanEval 上降低约 21.65%。AQuA 上本文方法相比 Fullconnect 仍降低约 24.23%，但相对 Agentprune 的成本优势不明显，说明在部分多项选择任务中，消息筛选策略仍有进一步优化空间。



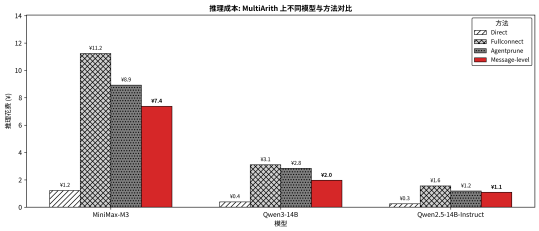
(a) MMLU 成本



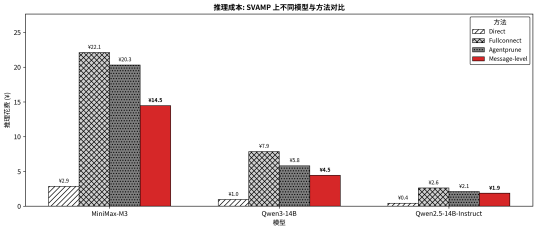
(b) GSM8K 成本



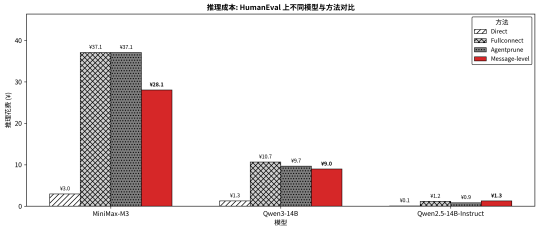
(c) AQuA 成本



(d) MultiArith 成本



(e) SVAMP 成本



(f) HumanEval 成本

图 3-4 各数据集上不同模型的推理成本比较

图 3-4进一步展示了各数据集上的成本差异。Direct 方法消耗最低，但缺少多智能体之间的信息交互；Fullconnect 方法在各数据集上普遍带来更高成本，反映出全连接通信结构的 token 显著增加的问题；Agentprune 能够通过剪枝降低部分开销，但不同任务上的成本和准确率表现仍存在波动。相比之下，本文方法通常能够在保留任务准确率的同时降低 Fullconnect 带来的额外成本，体现出更好的综合性价比。

3.2.3 小模型超过大模型的可能性分析

除比较不同方法的整体准确率和推理成本外，本文进一步关注一个实际问题：在引入消息压缩后，较小规模模型是否可能接近甚至超过更大规模模型。该问题对于多智能体系统部署具有现实意义，因为小模型通常具有更低的调用成本；如果通信优化能够弥补部分模型能力差距，则系统可以在成本受限场景中获得更高的性价比。

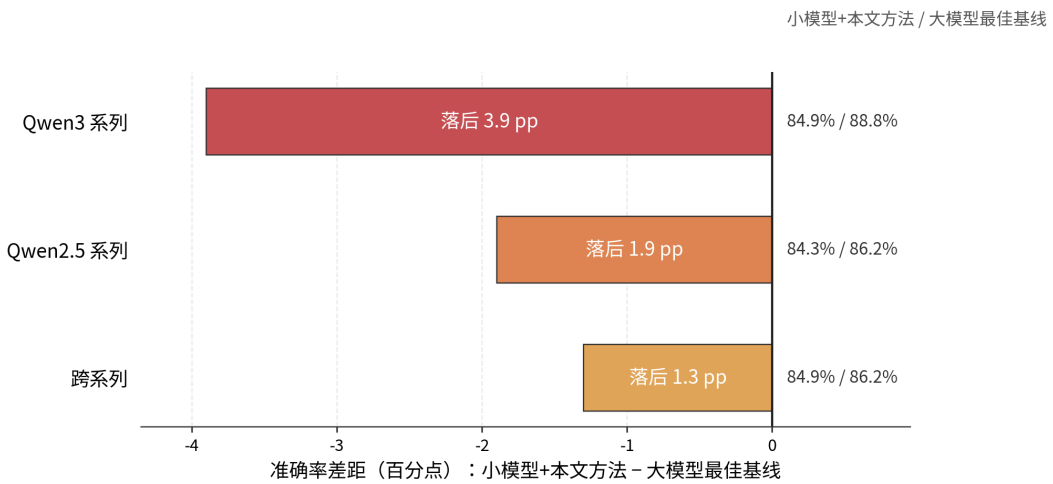


图 3-5 MMLU 上小模型相对大模型最佳基线的准确率差距

图 3-5以 MMLU 为例，展示小模型采用本文方法后相对大模型最佳基线的准确率差值，其中负值表示小模型仍低于对应的大模型基线。结果显示，小模型 + 本文方法尚未在该数据集上超过大模型最佳基线，但在 Qwen3 系列、Qwen2.5 系列和跨系列对比中，差距分别为 3.90、1.90 和 1.30 个百分点。也就是说，消息压缩不能直接消除模型规模带来的基础能力差距，但可以在一定程度上降低小模型与大模型之间的性能间隔。

从机制上看，小模型更容易受到冗余上下文、重复推理和无关消息的干扰，因此消息压缩带来的收益往往更明显。当任务主要依赖可提取的关键信息、标准化推理步骤或多智能体之间的互补观点时，小模型有机会通过更高质量的通信上下文接近大模型表现，甚至在部分强噪声或高成本通信场景中超过大模型的低效协作基线。相反，对于 AQuA、HumanEval 等更依赖复杂选项辨析或可执行验证的任务，仅压缩消息仍不足以稳定弥补模型能力差距。

因此，本阶段实验更支持：小模型通过通信优化具备接近大模型的潜力，但尚不能认为可以普遍超过大模型。后续若要进一步验证“小模型超过大模型”的可能性，需要在更多数据集上加入小模型 + 压缩方法与大模型 Direct、Fullconnect、Agentprune 等基线的成对比较，并结合成本指标判断其是否形成真正的准确率–成本优势。

综合推理性能、推理成本和模型规模三个维度来看，本文方法的目标是在多智能体协作场景下实现更合理的准确率–成本平衡。实验结果表明，消息级协作能够在减少冗余通信的同时保持多智能体推理收益，为构建更高效的多智能体大模型推理系统

提供了一种可行方向。

第四章　结论

4.1　研究进度与规划

第一阶段：前期调研与问题动机分析（第 1–3 个月，已完成）：

1. 文献综述与项目背景分析：系统调研多智能体协作、大语言模型推理、多智能体系统通信优化以及上下文压缩等相关工作，重点分析多智能体系统中消息传递带来的上下文冗余、token 开销增加以及准确率下降等问题。同时，阅读并分析 AgentPrune 项目中智能体节点、通信边执行流程的实现方式，为后续方法设计奠定基础。

2. 前期实验验证与算法动机探索：在 MMLU 任务上运行原始多智能体框架，观察不同智能体之间的消息传递模式和 token 消耗情况。通过实验发现，多个智能体在推理过程中往往会产生大量相似 reasoning 内容，而下游智能体真正需要的是与自身角色相关的关键信息和最终答案。因此，本课题提出在多智能体通信过程中引入消息压缩机制，在尽量保持任务准确率的前提下降低通信成本和上下文长度。

第二阶段：消息压缩框架设计与实现（第 4–6 个月，已完成）：

1. 消息结构化与压缩接口设计：为了解决原始输出中 reasoning 与 answer 混杂的问题，设计了结构化消息表示方式，通过区分结构化输出与普通文本输出，并在结构化消息中显式保留答案和推理字段。基于该结构，设计统一的消息压缩配置入口和命令行开关，使不同压缩模块可以独立启用或关闭，并保持默认行为与原始基线一致。

2. 核心压缩模块实现：实现了三个消息压缩模块。首先，角色感知压缩模块根据接收方智能体的角色，对前驱节点传来的推理过程按句切分并进行相关性过滤，保留与当前角色更相关的推理内容，同时完整保留 answer。其次，节点感知压缩模块对同一节点收到的多条前驱消息进行跨边聚合，通过句向量相似度检测冗余内容，并为保留句子添加角色标签，减少重复推理信息。最后，分歧感知压缩模块在每轮执行结束后，根据多个智能体答案的一致性生成纯字符摘要，将共识或分歧信息写入节点历史记忆中，从而降低后续轮次的上下文负担。

第三阶段：算法优化与性能分析（第 7–9 个月，进行中）：

1. 压缩策略优化：目前正在进一步分析不同任务、不同智能体角色和不同图结构下的消息冗余分布，优化角色相关性过滤阈值、跨节点聚合策略以及摘要生成规则。重点关注压缩率、准确率、推理成本和运行时间之间的平衡，避免过度压缩导致关键

信息丢失，也避免压缩不足导致 token 节省效果不明显。

2. 多数据集实验分析：在 MMLU、GSM8K、HumanEval、AQuA、SVAMP、MultiArith 等数据集上进行实验，比较原始 AgentPrune、多智能体全连接模式以及本文消息压缩方法的表现。

第四阶段：最终实现与论文撰写（第 10–12 个月）：

1. 最终实现与系统完善：在现有原型基础上继续完善消息压缩框架，提升其在不同任务和模型上的稳定性。进一步整理代码结构、实验脚本和配置参数，保证方法能够方便复现，并补充必要的测试用例，降低后续扩展和维护成本。

2. 论文撰写与成果总结：整理实验结果，系统总结消息压缩方法的设计动机、实现细节、性能收益和局限性。重点阐述本方法如何在多智能体通信过程中减少冗余 reasoning、降低 token 成本，并保持较好的任务性能。最终完成毕业论文撰写、修改和答辩准备工作。

4.2 研究过程中遇到的问题

在研究过程中，本课题遇到的困难、问题及解决措施如下：

问题 1：原项目中不同大语言模型接口返回的结果格式不统一，有的返回普通字符串，有的返回包含 reasoning、answer 等字段的结构化对象。

解决方法：在节点执行结果进入消息传递流程前增加统一的输出解析逻辑，对不同来源的模型返回结果进行格式规范化。对于能够解析出推理过程和最终答案的结果，统一封装为结构化消息；对于无法解析的普通文本，则保留为非结构化消息，并在压缩模块中进行显式判断，避免错误处理。

问题 2：原有代码在节点异步执行和消息传递过程中会将部分模型返回结果压平成普通文本，导致后续模块无法区分 reasoning 和 answer，也无法判断当前消息是否可以被安全压缩。

解决方法：在消息传递的数据结构中引入提示词约束，保证输出为结构化消息。

问题 3：消息压缩模块需要插入到原有多智能体图执行流程中，但原项目中节点执行、空间边消息读取、时间记忆读取和图轮次更新之间耦合较多，直接修改容易影响 AgentPrune 的基线行为。

解决方法：采用开关式集成方式，将压缩配置集中到统一配置类中，并通过命令

行参数控制是否启用。默认情况下所有压缩模块关闭，使系统行为与原始代码保持一致；只有显式打开对应参数时，才在节点读取前驱消息或图执行轮次结束时触发压缩逻辑，从而降低对原有实验流程的影响。

问题 4：在多个实验脚本中加入消息压缩参数时，需要保证 MMLU、GSM8K、HumanEval、AQuA、SVAMP、MultiArith 等不同任务入口的参数名称、默认值和配置传递方式一致，否则容易出现某些数据集可以运行、某些数据集参数无效的问题。

解决方法：将消息压缩相关参数封装为统一函数，在各个实验入口中复用同一套参数添加和配置构建流程。这样可以避免在不同任务中重复编写代码，也方便后续统一修改开关名称和默认行为。

参考文献

[1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.

[2] ACHIAM J, ADLER S, AGARWAL S, et al. Gpt-4 technical report[J]. arXiv preprint arXiv:2303.08774, 2023.

[3] TOUVRON H, LAVRIL T, IZACARD G, et al. Llama: Open and efficient foundation language models[J]. arXiv preprint arXiv:2302.13971, 2023.

[4] TOUVRON H, MARTIN L, STONE K, et al. Llama 2: Open foundation and fine-tuned chat models[J]. arXiv preprint arXiv:2307.09288, 2023.

[5] GRATTAFIORI A, DUBEY A, JAUHRI A, et al. The llama 3 herd of models[J]. arXiv preprint arXiv:2407.21783, 2024.

[6] BAI J, BAI S, CHU Y, et al. Qwen technical report[J]. arXiv preprint arXiv:2309.16609, 2023.

[7] HUI B, YANG J, CUI Z, et al. Qwen2.5-coder technical report[J]. arXiv preprint arXiv:2409.12186, 2024.

[8] YANG A, LI A, YANG B, et al. Qwen3 technical report[J]. arXiv preprint arXiv:2505.09388, 2025.

[9] QIAN C, LIU W, LIU H, et al. Chatdev: Communicative agents for software development[C]//Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers). 2024: 15174-15186.

[10] BOIKO D A, MACKNIGHT R, KLINE B, et al. Autonomous chemical research with large language models[J]. Nature, 2023, 624(7992): 570-578.

[11] DRIESS D, XIA F, SAJJADI M S M, et al. Palm-e: An embodied multimodal language model[J]. arXiv preprint arXiv:2303.03378, 2023.

[12] XI Z, CHEN W, GUO X, et al. The rise and potential of large language model based agents: A survey[J]. Science China Information Sciences, 2025, 68(2): 121101.

[13] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems, 2022, 35: 24824-24837.

[14] WANG X, WEI J, SCHUURMANS D, et al. Self-consistency improves chain of thought reasoning in language models[J]. arXiv preprint arXiv:2203.11171, 2022.

[15] YAO S, YU D, ZHAO J, et al. Tree of thoughts: Deliberate problem solving with large language models[J]. Advances in neural information processing systems, 2023, 36: 11809-11822.

[16] BESTA M, BLACH N, KUBICEK A, et al. Graph of thoughts: Solving elaborate problems with large language models[C]//Proceedings of the AAAI conference on artificial intelligence. 2024, 38(16): 17682-17690.

[17] SCHICK T, DWIVEDI-YU J, DESSÌ R, et al. Toolformer: Language models can teach themselves to use tools[J]. Advances in neural information processing systems, 2023, 36: 68539-68551.

[18] QIN Y, LIANG S, YE Y, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis[C]//International Conference on Learning Representations. 2024, 2024: 9695-9717.

[19] WANG L, XU W, LAN Y, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models[C]//Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers). 2023: 2609-2634.

[20] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 186345.

[21] LIU Z, BAI X, CHEN K, et al. A survey on the feedback mechanism of LLM-based AI agents[C]//Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence. International Joint Conferences on Artificial Intelligence, 2025: 10582-10592.

[22] CHENG M, WANG D, YU S, et al. A Comprehensive Survey of the LLM-Based Agent: The Contextual Cognition Perspective[J]. 2026.

[23] YAO S, ZHAO J, YU D, et al. React: Synergizing reasoning and acting in language models[J]. arXiv preprint arXiv:2210.03629, 2022.

[24] AHN M, BROHAN A, BROWN N, et al. Do as i can, not as i say: Grounding language in robotic affordances[J]. arXiv preprint arXiv:2204.01691, 2022.

[25] SHINN N, CASSANO F, GOPINATH A, et al. Reflexion: Language agents with verbal reinforcement learning[J]. Advances in neural information processing systems, 2023, 36: 8634-8652.

[26] PARK J S, O'BRIEN J, CAI C J, et al. Generative agents: Interactive simulacra of human behavior[C]//Proceedings of the 36th annual acm symposium on user interface software and technology. 2023: 1-22.

[27] WANG G, XIE Y, JIANG Y, et al. Voyager: An open-ended embodied agent with large language models[J]. arXiv preprint arXiv:2305.16291, 2023.

[28] LAZARIDOU A, KUNCORO A, GRIBOVSKAYA E, et al. Mind the gap: Assessing temporal generalization in neural language models[J]. Advances in Neural Information Processing Systems, 2021, 34: 29348-29363.

[29] LIU N F, LIN K, HEWITT J, et al. Lost in the middle: How language models use long contexts[J]. Transactions of the association for computational linguistics, 2024, 12: 157-173.

[30] BAI Y, LV X, ZHANG J, et al. Longbench: A bilingual, multitask benchmark for long context understanding[C]//Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers). 2024: 3119-3137.

[31] DU Y, LI S, TORRALBA A, et al. Improving factuality and reasoning in language models through multiagent debate[J]. arXiv preprint arXiv:2305.14325, 2023.

[32] GUO T, CHEN X, WANG Y, et al. Large language model based multi-agents: A survey of progress and challenges[J]. arXiv preprint arXiv:2402.01680, 2024.

[33] TALEBIRAD Y, NADIRI A. Multi-agent collaboration: Harnessing the power of intelligent llm agents[J]. arXiv preprint arXiv:2306.03314, 2023.

[34] LIANG T, HE Z, JIAO W, et al. Encouraging divergent thinking in large language models through multi-agent debate[C]//Proceedings of the 2024 conference on empirical methods in natural language processing. 2024: 17889-17904.

[35] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: Meta programming for a multi-agent collaborative framework[C]//International Conference on Learning Representations. 2024, 2024: 23247-23275.

[36] WU Q, BANSAL G, ZHANG J, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation[J]. arXiv preprint arXiv:2308.08155, 2023.

[37] LI G, HAMMOUD H, ITANI H, et al. Camel: Communicative agents for" mind" exploration of large language model society[J]. Advances in neural information processing systems, 2023, 36: 51991-52008.

[38] WEI T, LI T W, LIU Z, et al. Agentic reasoning for large language models[J]. arXiv preprint arXiv:2601.12538, 2026.

[39] LIU Z, ZHANG Y, LI P, et al. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization[J]. arXiv preprint arXiv:2310.02170, 2023.

[40] HU S, SHEN L, ZHANG Y, et al. Learning multi-agent communication from graph modeling perspective[J]. arXiv preprint arXiv:2405.08550, 2024.

[41] ZHANG G, YUE Y, SUN X, et al. G-designer: Architecting multi-agent communication topologies via graph neural networks[J]. arXiv preprint arXiv:2410.11782, 2024.

[42] ZENG X, SU H, WANG Z, et al. Graph diffusion for robust multi-agent coordination[C]//Forty-second International Conference on Machine Learning. 2025.

[43] ZHANG G, YUE Y, LI Z, et al. Cut the crap: An economical communication pipeline for llm-based multi-agent systems[C]//International Conference on Learning Representations. 2025, 2025: 75389-75428.

[44] WANG Z, WANG Y, LIU X, et al. Agentdropout: Dynamic agent elimination for token-efficient and high-performance llm-based multi-agent collaboration[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025: 24013-24035.

[45] ZHANG R, ZHAO X, WANG R, et al. Safesieve: From heuristics to experience in progressive pruning for llm-based multi-agent communication[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2026, 40(35): 29892-29900.

[46] LI B, ZHAO Z, LEE D H, et al. Adaptive graph pruning for multi-agent communication[J]. arXiv preprint arXiv:2506.02951, 2025.

[47] WANG S, ZHANG G, YU M, et al. G-safeguard: A topology-guided security lens and treatment on llm-based multi-agent systems[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025: 7261-7276.

[48] ZHUGE M, WANG W, KIRSCH L, et al. Gptswarm: Language agents as optimizable graphs[C]//Forty-first International Conference on Machine Learning. 2024.

[49] WANG S, LU R, YANG Z, et al. Agentconductor: Topology evolution for multi-agent competition-level code generation[J]. arXiv preprint arXiv:2602.17100, 2026.

[50] ZHANG J, XIANG J, YU Z, et al. Automating agentic workflow generation, 2024a[J]. URL https://arxiv. org/abs/2410.10762.

[51] ZHOU H, WAN X, SUN R, et al. Multi-agent design: Optimizing agents with better prompts and topologies[J]. arXiv preprint arXiv:2502.02533, 2025.

[52] ZHANG G, NIU L, FANG J, et al. Multi-agent architecture search via agentic supernet[J]. arXiv preprint arXiv:2502.04180, 2025.

[53] LEONG H Y, WU Y. DynaSwarm: Dynamically graph structure selection for llm-based multi-agent system[J]. arXiv preprint arXiv:2507.23261, 2025.

[54] LI Z, LIU Y, SU Y, et al. Prompt compression for large language models: A survey[C]//Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). 2025: 7182-7195.

[55] LI Y, DONG B, GUERIN F, et al. Compressing context to enhance inference efficiency of large language models[C]//Proceedings of the 2023 conference on empirical methods in natural language processing. 2023: 6342-6353.

[56] JIANG H, WU Q, LIN C Y, et al. Llmlingua: Compressing prompts for accelerated inference of large language models[C]//Proceedings of the 2023 conference on empirical methods in natural language processing. 2023: 13358-13376.

[57] PAN Z, WU Q, JIANG H, et al. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression[C]//Findings of the Association for Computational Linguistics: ACL 2024. 2024: 963-981.

[58] SHANDILYA S, XIA M, GHOSH S, et al. Taco-rl: Task aware prompt compression optimization with reinforcement learning[C]//Findings of the Association for Computational Linguistics: ACL 2025. 2025: 1582-1597.

[59] CHUANG Y N, XING T, CHANG C Y, et al. Learning to compress prompt in natural language formats[C]//Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). 2024: 7756-7767.

[60] LAI X, XU W, YANG Y, et al. MiniMax Sparse Attention[J]. arXiv preprint arXiv:2606.13392, 2026.

[61] HENDRYCKS D, BURNS C, BASART S, et al. Measuring massive multitask language understanding[J]. arXiv preprint arXiv:2009.03300, 2020.

[62] COBBE K, KOSARAJU V, BAVARIAN M, et al. Training verifiers to solve math word problems[J]. arXiv preprint arXiv:2110.14168, 2021.

[63] LING W, YOGATAMA D, DYER C, et al. Program induction by rationale generation: Learning to solve and explain algebraic word problems[C]//Proceedings of the 55th annual meeting of the association for computational linguistics (volume 1: Long papers). 2017: 158-167.

[64] ROY S, ROTH D. Solving general arithmetic word problems[C]//Proceedings of the 2015 conference on empirical methods in natural language processing. 2015: 1743-1752.

[65] PATEL A, BHATTAMISHRA S, GOYAL N. Are NLP models really able to solve simple math word problems?[C]//Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies. 2021: 2080-2094.

[66] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code[J]. arXiv preprint arXiv:2107.03374, 2021.